



Classical NLP Techniques & ML Models



1. What is this topic?

Think of NLP like teaching a robot 🤖 to **read and understand language like a human**.

Once text is cleaned and converted into numbers, we now want the machine to:

- 🧩 Understand grammar
- 🏷️ Identify important names
- 📁 Classify text
- 🗨️ Discover hidden patterns



In simple words:

Classical NLP = **Basic intelligence tasks we teach machines to understand text**



2. Why do we need this? (Problem → Solution)

● Problem:

Raw text is just words...

Machines don't know:

- What is a noun?
 - What is a person name?
 - What topic is being discussed?
-



✓ Solution:

We apply **NLP techniques + ML models** to extract meaning

3. Core Concepts

◆ 1. Part-of-Speech (POS) Tagging

What is it?

Identifying the **role of each word** in a sentence

Example:

"I love NLP"

Output:

I → Pronoun

love → Verb

NLP → Noun

Analogy:

Like labeling parts of a sentence in school grammar class 

Why important?

- Helps understand sentence structure
 - Useful in chatbots, translation
-

◆ 2. Named Entity Recognition (NER)

💡 What is it?

Extracting **important real-world entities**

📦 Example:

"Elon Musk founded SpaceX"

Output:

Elon Musk → Person

SpaceX → Organization

💡 Analogy:

Highlighting important names in a paragraph 🖋️

💡 Use cases:

- Resume parsing
 - Healthcare (disease, drug names)
 - News analysis
-

◆ 3. Text Classification

🗨️ What is it?

Classifying text into categories

📦 Examples:

Sentiment Analysis

"I love this product" → Positive

"I hate this" → Negative

Spam Detection

"Win \$1000 now!!!" → Spam

🗨️ Analogy:

Like sorting emails into folders 📁

🤖 Models Used:

◆ Naive Bayes

- Simple, fast
- Works well for text

◆ Logistic Regression

- Most commonly used
- Good baseline

◆ Support Vector Machine (SVM)

- Powerful for classification
 - Works well in high-dimensional data
-

◆ 4. Topic Modeling

🗨️ What is it?

Finding **hidden topics** in large text data

📦 Example:

Documents:

Doc1: "football match goal"

Doc2: "AI machine learning model"

👉 Model detects:

- Topic 1 → Sports
 - Topic 2 → Technology
-

🔧 Technique:

◆ Latent Dirichlet Allocation (LDA)

- Groups words into topics
 - Finds patterns automatically
-

🗨️ Analogy:

Like grouping books in a library 📖 by topic

4. How Everything Fits Together

Text → Preprocessing → Representation → NLP Task → Model → Output

Example Flow:

"I love this product"

↓

POS → identify structure

↓

NER → detect entities

↓

Classification → Positive sentiment

↓

Topic Modeling → Product feedback

5. Real-World Use Cases (VERY IMPORTANT FOR YOU)

Healthcare

- Extract diseases (NER)
 - Analyze doctor notes (classification)
-

Customer Support

- Classify tickets (urgent / normal)
 - Detect customer sentiment
-

Data Engineering Pipelines

S3 → Preprocessing → NLP Tasks → ML Model → Insights Dashboard

Final Mental Model

 Classical NLP = Understanding structure + meaning + category of text

Interview Tip (VERY IMPORTANT)

If asked:

👉 “What are common NLP techniques?”

Answer:

- POS Tagging → grammar understanding
 - NER → entity extraction
 - Text Classification → prediction
 - Topic Modeling → pattern discovery
-

PRACTICAL IMPLEMENTATION — Classical NLP Techniques & ML Models

Now let's implement **ALL core techniques in Python** step-by-step 🔥
(Exactly what interviewers expect)

1. Part-of-Speech (POS) Tagging

```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

text = "I love NLP"
tokens = word_tokenize(text)

pos_tags = pos_tag(tokens)

print(pos_tags)
```

Output

[('I', 'PRP'), ('love', 'VBP'), ('NLP', 'NN')]

Explanation

- **PRP** → Pronoun
- **VBP** → Verb
- **NN** → Noun

 Machine now understands grammar

2. Named Entity Recognition (NER)

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "Elon Musk founded SpaceX in California"
doc = nlp(text)

for ent in doc.ents:
    print(ent.text, ent.label_)
```

Output

Elon Musk PERSON
SpaceX ORG
California GPE

Explanation

- **PERSON** → human name
 - **ORG** → organization
 - **GPE** → location
-



3. Text Classification (Sentiment Analysis)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

texts = ["I love this", "I hate this"]
labels = [1, 0]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(texts)

model = LogisticRegression()
model.fit(X, labels)

test = vectorizer.transform(["I love NLP"])
print(model.predict(test))
```

Output

[1] # Positive

Explanation

- Text → TF-IDF → Numbers
 - Model learns → Positive / Negative
-



4. Topic Modeling (LDA)

```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

docs = [
    "football match goal",
    "AI machine learning",
    "deep learning neural network"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(docs)

lda = LatentDirichletAllocation(n_components=2)
lda.fit(X)

print(lda.components_)
```

Explanation

- Groups words into topics
- Finds hidden themes



5. Real-World Use Cases (Interview Gold)



Healthcare

- NER → extract disease names
- Classification → patient sentiment



Customer Support

- Classify tickets
- Detect urgency



Resume Screening

- NER → extract skills, companies
-



Data Engineering Pipeline (Your Strong Area)

S3 → Glue → NLP Processing → ML Model → Snowflake Dashboard



6. Mini Project



Project: "Smart Customer Support Analyzer"



Goal:

- Extract entities (names, products)
 - Detect sentiment
 - Classify issue type
-



Steps:

1. Input customer query
 2. Preprocess text
 3. Apply:
 - POS tagging
 - NER
 - Sentiment classification
 4. Output structured result
-



Tools:

- Python
- NLTK
- spaCy
- Scikit-learn

7. Common Mistakes (VERY IMPORTANT)

1. Skipping preprocessing

 Garbage input = bad results

2. Using wrong model

 Start with Logistic Regression

3. Ignoring evaluation

 Always check accuracy

4. Overcomplicating early

 Master basics first

5. Not understanding outputs

 Always interpret model results



Final Practical Understanding

Text → Clean → Represent → Apply NLP Task → Model → Insight
